Este documento estabelece a definição formal e definitiva da arquitetura de software, stack tecnológica e divisão de responsabilidades para o desenvolvimento da plataforma **Reaproveita**, servindo como documento de referência a partir da Sprint 1.

1. Visão Geral da Arquitetura

O sistema adota o padrão **BFF (Backend-for-Frontend)** integrado via **Next.js (App Router)**, desacoplando o cliente de acessos diretos ao banco de dados e centralizando as regras de negócio em ambiente de execução seguro no servidor (Node.js runtime). A persistência e os serviços de infraestrutura gerenciada ficam a cargo do **Supabase**.

Plaintext

```
[Cliente Web / PWA]
       │
       │ (Chamadas HTTP / Server Actions via HTTPS)
       ▼
[Next.js — Camada de Aplicação & BFF]
  ├── Server Actions & Route Handlers (Regras de negócio, validações Zod e ACID)
  └── Supabase SSR Client (Gestão transparente de sessão em Cookies HttpOnly)
       │
       │ (Conexão transacional direta / RPC / PostGIS)
       ▼
[Supabase — Infraestrutura & Dados]
  ├── PostgreSQL 15+ com PostGIS (Consultas espaciais e consistência de dados)
  ├── Supabase Auth (Emissão e controle de identidade)
  ├── pg_cron (Agendamento interno de expiração de lotes e reservas)
  └── Supabase Storage (Imagens de produtos e comprovantes de estabelecimentos)
```

2. Matriz de Componentes e Responsabilidades

Camada / Domínio	Tecnologia Selecionada	Responsabilidade no Sistema
Frontend & BFF	Next.js 14+ (App Router) + TypeScript	Interface responsiva (PWA), renderização híbrida (SSR/Client Components), rotas de API protegidas (Route Handlers) e execução de mutações de dados no servidor via Server Actions.
Estilização	Tailwind CSS	Sistema de design declarativo, focado em responsividade e interfaces otimizadas para uso móvel em campo.
Validação de Schemas	Zod	Validação estrita de contratos de entrada em todas as Server Actions e webhooks externos antes de qualquer processamento.

Camada / Domínio	Tecnologia Selecionada	Responsabilidade no Sistema
Banco de Dados Relacional	PostgreSQL (via Supabase)	Persistência transacional com conformidade ACID, integridade referencial e controle de concorrência em nível de linha (FOR UPDATE).
Motor Geoespacial	PostGIS (Extensão Postgres)	Armazenamento de coordenadas geográficas (GEOMETRY(Point, 4326)) e consultas espaciais indexadas via GiST para cálculo de proximidade (ST_DWithin).
Autenticação & Sessão	Supabase Auth + @supabase/ssr	Gerenciamento de credenciais e controle de acesso baseado em papéis (RBAC: Comerciante, Beneficiário, Admin). Sessão mantida estritamente em cookies HttpOnly, Secure e SameSite=Lax.
Jobs e Ciclo de Vida (TTL)	pg_cron (Extensão Postgres)	Execução programada dentro do banco de dados (a cada minuto) para expirar lotes não resgatados e atualizar estados de reservas.
Mapas & Interface Visual	Mapbox GL (ou MapLibre GL)	Renderização vetorial interativa de mapa no cliente para visualização e busca de pontos de coleta.
Pagamentos (Pix)	AbacatePay	Processamento de transações Pix com confirmação assíncrona orientada a eventos via webhooks idempotentes.
E-mails Transacionais	Resend	Disparo de comunicações de autenticação (magic links/confirmações) e notificações críticas de reserva de pedidos.
Notificações em Tempo Real	OneSignal (Web Push)	Notificações push direcionadas ao navegador/PWA para alertas de novos lotes disponíveis e avisos de coleta próxima.
Deploy & Observabilidade	Vercel + Sentry	Hospedagem com suporte nativo a Edge/Serverless functions e rastreamento contínuo de erros e exceções não tratadas.

3. Diretrizes Arquiteturais e Regras de Implementação

1. **Vedação de Acesso Direto de Escrita pelo Cliente:**

É estritamente proibido utilizar o SDK do Supabase no cliente (*browser*) para mutações diretas em tabelas críticas (`INSERT` , `UPDATE` , `DELETE` em lotes, estoques e reservas). Toda mutação deve trafegar por *Server Actions* ou *Route Handlers* no Next.js, onde validações de permissão e regras de concorrência são executadas.

2. **Prevenção de Dupla Reserva (Race Conditions):**

A baixa de estoque de alimentos deve ser executada atomicamente no banco de dados através de funções em PL/pgSQL (*Postgres RPC*) ou transações com bloqueio pessimista (`SELECT ... FOR`

`UPDATE`), impedindo que dois usuários simultâneos reservem a mesma unidade disponível.

3. **Isolamento de Segurança de Credenciais:**

Tokens de acesso e chaves administrativas (`service_role`) jamais devem ser expostos no código client-side. A comunicação com o Supabase no navegador opera exclusivamente com a `anon_key` e o contexto da sessão autenticada em cookie seguro.

4. **Tratamento de Webhooks Externos:**

Endpoints que recebam confirmações de pagamento (AbacatePay) devem implementar chave de idempotência gravada no PostgreSQL, garantindo que retransmissões de rede não gerem confirmações duplicadas de retirada.

4. Decisões de Escopo e Exclusões da Sprint 1

- **Redis:** Descartado para a fase inicial. O cache em memória e o gerenciamento de filas externas introduziriam sobrecarga desnecessária de infraestrutura; o PostgreSQL em conjunto com o `pg_cron` atende a toda a demanda volumétrica e temporal do protótipo acadêmico.
- **Aplicativo Mobile Nativo:** Substituído integralmente pelo modelo Web Responsivo/PWA no Next.js, unificando a base de código e eliminando a dependência de submissão a lojas de aplicativos durante o ciclo de sprints.